

Usulan Ide Tugas Akhir

NIM	2305551013
Nama	Agus Arya Wiraguna
Bidang Keahlian	Network and Cloud Computing
Topik Networking dan Cloud Computing	Network & Server Monitoring/Management
Judul	Rancang Bangun Sistem Monitoring dan Manajemen Hybrid Cloud Berbasis Web Menggunakan OpenStack API, React.js, Golang, dan Docker.
Dosen Pembimbing	1. Dr.Eng. I Made Agus Dwi Suarjaya, ST, M.Eng. 2. Ir. I Nyoman Piarsa, ST., MT., IPM.

1. Permasalahan

Perkembangan teknologi hybrid cloud mendorong organisasi untuk mengombinasikan private cloud dan public cloud guna meningkatkan fleksibilitas, skalabilitas, serta efisiensi dalam pengelolaan infrastruktur teknologi informasi (Saxena, Gupta, & Singh, 2021). Penggunaan beberapa cloud provider secara bersamaan juga meningkatkan kompleksitas pengelolaan karena setiap provider memiliki mekanisme autentikasi, struktur API, model sumber daya, serta antarmuka administrasi yang berbeda (Nzanzu, et al., 2022). Perbedaan tersebut menyebabkan administrator harus mengelola beberapa platform secara terpisah sehingga proses monitoring dan pengelolaan sumber daya menjadi kurang efisien (Saxena, Gupta, & Singh, 2021).

Penelitian mengenai arsitektur manajemen sumber daya cloud menunjukkan bahwa heterogenitas platform menjadi salah satu tantangan utama dalam membangun sistem monitoring dan pengelolaan cloud yang terintegrasi (Andreazi, et al., 2021). Selain itu, implementasi lingkungan multi-cloud masih menghadapi tantangan pada aspek monitoring terpadu, interoperabilitas antar cloud provider, otomatisasi pengelolaan sumber daya, serta penyediaan mekanisme observabilitas yang konsisten (Waseem, et al., 2024). Akibatnya, administrator belum memperoleh informasi kondisi infrastruktur secara menyeluruh melalui satu platform terpusat (Nzanzu, et al., 2022).

Berdasarkan permasalahan tersebut, diperlukan sebuah platform berbasis web yang mampu mengintegrasikan OpenStack, Google Cloud Platform, dan Microsoft Azure ke dalam satu dashboard terpusat melalui mekanisme Provider Abstraction Layer, Resource Synchronization, dan Unified Data Model sehingga informasi dari berbagai cloud dapat disajikan secara konsisten. Sistem yang diusulkan diharapkan mampu melakukan monitoring kondisi cloud, sinkronisasi sumber daya, serta manajemen *lifecycle* virtual machine melalui antarmuka yang seragam sehingga proses pengelolaan lingkungan hybrid cloud menjadi lebih efektif dan efisien.

2. Tujuan Penelitian

Penelitian ini bertujuan untuk merancang dan mengembangkan sebuah Sistem Monitoring dan Manajemen Hybrid Cloud Berbasis Web yang mampu mengintegrasikan layanan OpenStack, Google Cloud Platform (GCP), dan Microsoft Azure ke dalam satu platform terpusat. Secara khusus, tujuan penelitian ini adalah sebagai berikut.

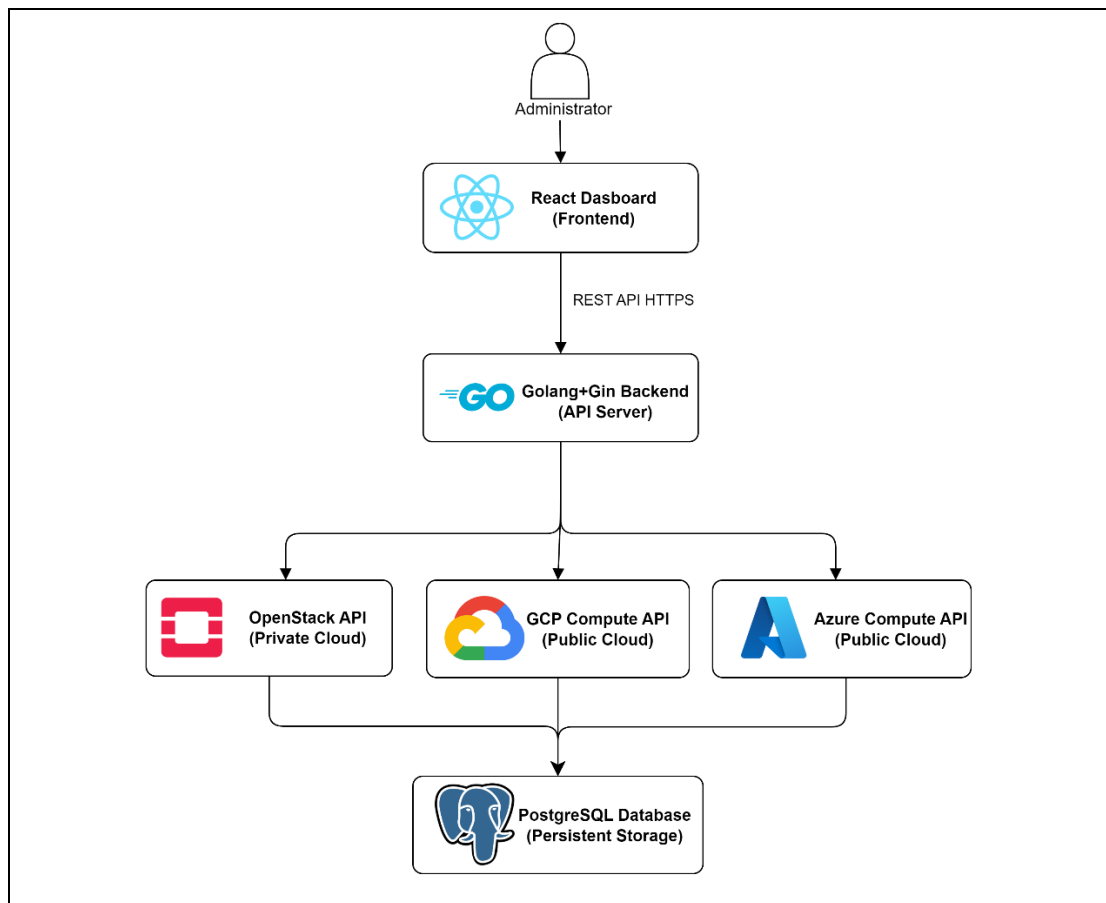
1. Merancang arsitektur sistem monitoring dan manajemen hybrid cloud berbasis web yang mampu mengintegrasikan private cloud dan public cloud dalam satu platform.
2. Mengembangkan Provider Abstraction Layer sebagai mekanisme integrasi antar cloud provider sehingga perbedaan implementasi API pada OpenStack, Google Cloud Platform, dan Microsoft Azure dapat dikelola melalui antarmuka yang seragam.
3. Mengembangkan mekanisme Resource Synchronization untuk memperoleh dan memperbarui informasi sumber daya dari setiap cloud provider secara terpusat.
4. Mengembangkan Unified Data Model untuk melakukan normalisasi metadata dari berbagai cloud provider sehingga informasi sumber daya dapat disimpan dan dikelola dalam struktur data yang konsisten.
5. Mengimplementasikan dashboard monitoring yang mampu menyajikan informasi kondisi cloud dan virtual machine secara terpusat, meliputi status layanan, status instance, serta informasi sumber daya dari seluruh cloud provider.

6. Mengimplementasikan fitur Virtual Machine Lifecycle Management yang meliputi operasi *create*, *start*, *stop*, dan *delete* virtual machine pada masing-masing cloud provider melalui satu antarmuka terpadu.

3. Gambaran Umum Penelitian

Sistem yang akan dikembangkan merupakan Sistem Monitoring dan Manajemen Hybrid Cloud Berbasis Web sebagai platform terpusat untuk melakukan monitoring dan pengelolaan sumber daya pada lingkungan hybrid cloud (Mamidala, et al., 2025). Sistem mengintegrasikan OpenStack sebagai private cloud dengan Google Cloud Platform (GCP) dan Microsoft Azure sebagai public cloud sehingga administrator dapat memantau dan mengelola sumber daya dari berbagai cloud provider melalui satu dashboard terpadu (Merlo, Fard, & Hawamdeh, 2025). Pendekatan integrasi antara private cloud dan public cloud telah banyak diterapkan pada implementasi hybrid cloud untuk memanfaatkan keunggulan masing-masing lingkungan komputasi (Mansouri, Prokhorenko, & Babar, 2020). Pendekatan ini sejalan dengan konsep multi-cloud management, yaitu penyediaan satu platform yang mampu mengelola berbagai cloud provider secara terpusat untuk meningkatkan efisiensi operasional (Li, Ma, Fang, He, & Li, 2025).

Sistem dirancang menggunakan arsitektur client-server dengan pendekatan layered architecture yang memisahkan fungsi antarmuka pengguna, logika bisnis, integrasi cloud provider, dan penyimpanan data ke dalam beberapa lapisan yang saling terhubung (Li, Ma, Fang, He, & Li, 2025). Pendekatan ini mendukung pengembangan sistem yang lebih modular, mudah dipelihara, serta memudahkan perluasan dukungan terhadap cloud provider lain pada masa mendatang. (Merlo, Fard, & Hawamdeh, 2025)



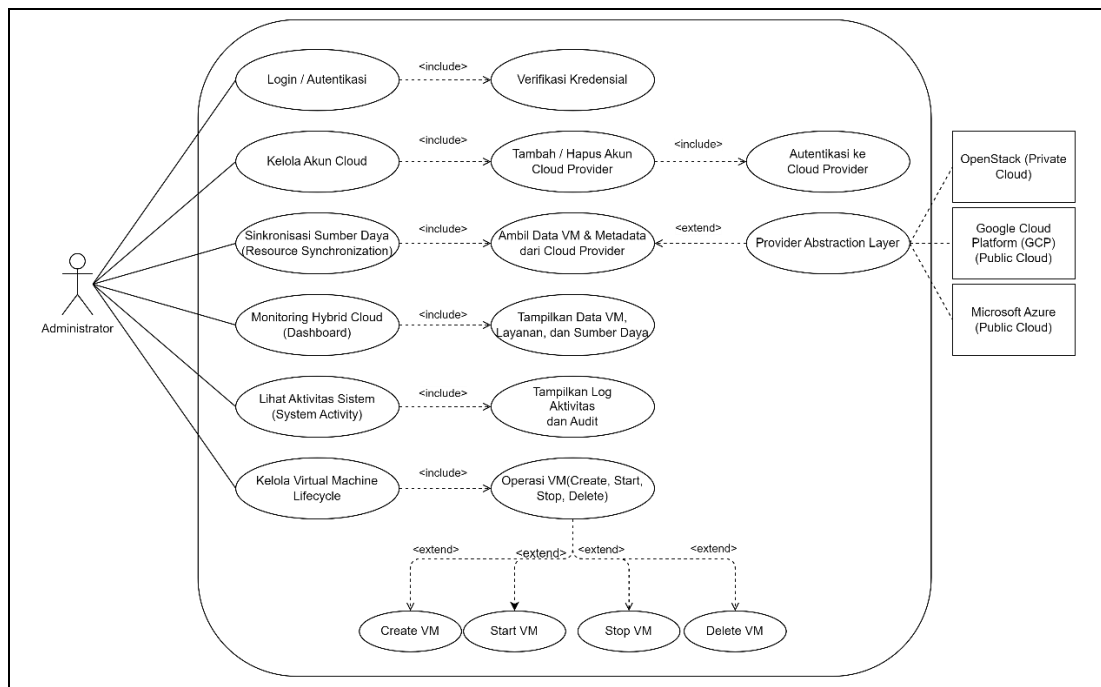
Gambar 1 Arsitektur Sistem Hybrid Cloud Monitoring dan Management.

Arsitektur sistem terdiri atas lima komponen utama yang saling terintegrasi, yaitu Frontend Dashboard, Backend Service, Provider Abstraction Layer, Database, dan Cloud Provider. Masing-masing komponen memiliki fungsi yang berbeda namun saling mendukung untuk membentuk sistem monitoring dan manajemen hybrid cloud yang terintegrasi, lima komponen utama, yaitu

1. Frontend Dashboard merupakan antarmuka utama yang digunakan administrator untuk berinteraksi dengan sistem. Komponen ini dikembangkan menggunakan React, TypeScript, Vite, Tailwind CSS, dan shadcn/ui sehingga mampu menyajikan informasi monitoring secara interaktif serta menyediakan antarmuka untuk melakukan pengelolaan virtual machine.
2. Backend Service dikembangkan menggunakan Golang dan Gin Framework sebagai pusat logika bisnis (*business logic*). Komponen ini bertanggung jawab menerima permintaan dari frontend, melakukan autentikasi, mengelola

komunikasi dengan cloud provider, melakukan sinkronisasi sumber daya, serta memproses setiap operasi manajemen virtual machine. (Andreazi, et al., 2021). Pada tahap autentikasi, backend mengelola proses verifikasi kredensial setiap cloud account sebelum layanan cloud diintegrasikan ke dalam sistem. Pengelolaan autentikasi yang aman diperlukan untuk menjaga keamanan akses terhadap infrastruktur cloud serta mengurangi risiko penyalahgunaan kredensial ketika sistem berkomunikasi dengan berbagai cloud provider (Alghofaili, Albattah, Alrajeh, Rassam, & Al-rimy, 2021).

3. Untuk mengatasi perbedaan implementasi API antar cloud provider, sistem menerapkan Provider Abstraction Layer. Komponen ini berfungsi sebagai lapisan integrasi yang menyembunyikan kompleksitas komunikasi dengan OpenStack, Google Cloud Platform, dan Microsoft Azure sehingga backend hanya berinteraksi melalui satu antarmuka yang seragam. Pendekatan provider abstraction terbukti mampu meningkatkan interoperabilitas sistem serta mempermudah pengembangan aplikasi yang mendukung lebih dari satu cloud provider (Andreazi, et al., 2021).
4. Seluruh informasi mengenai cloud account, metadata virtual machine, aktivitas sistem, serta hasil sinkronisasi sumber daya disimpan pada PostgreSQL sebagai basis data utama (Nzanzu, et al., 2022)
5. Seluruh komponen sistem dijalankan menggunakan Docker dan Docker Compose sehingga lingkungan pengembangan dan deployment menjadi lebih konsisten, portabel, serta mudah direplikasi pada berbagai lingkungan komputasi. Pemanfaatan teknologi container juga menjadi salah satu pendekatan yang banyak digunakan dalam implementasi lingkungan multi-cloud modern karena meningkatkan portabilitas aplikasi dan mempermudah proses deployment. (Waseem, et al., 2024)

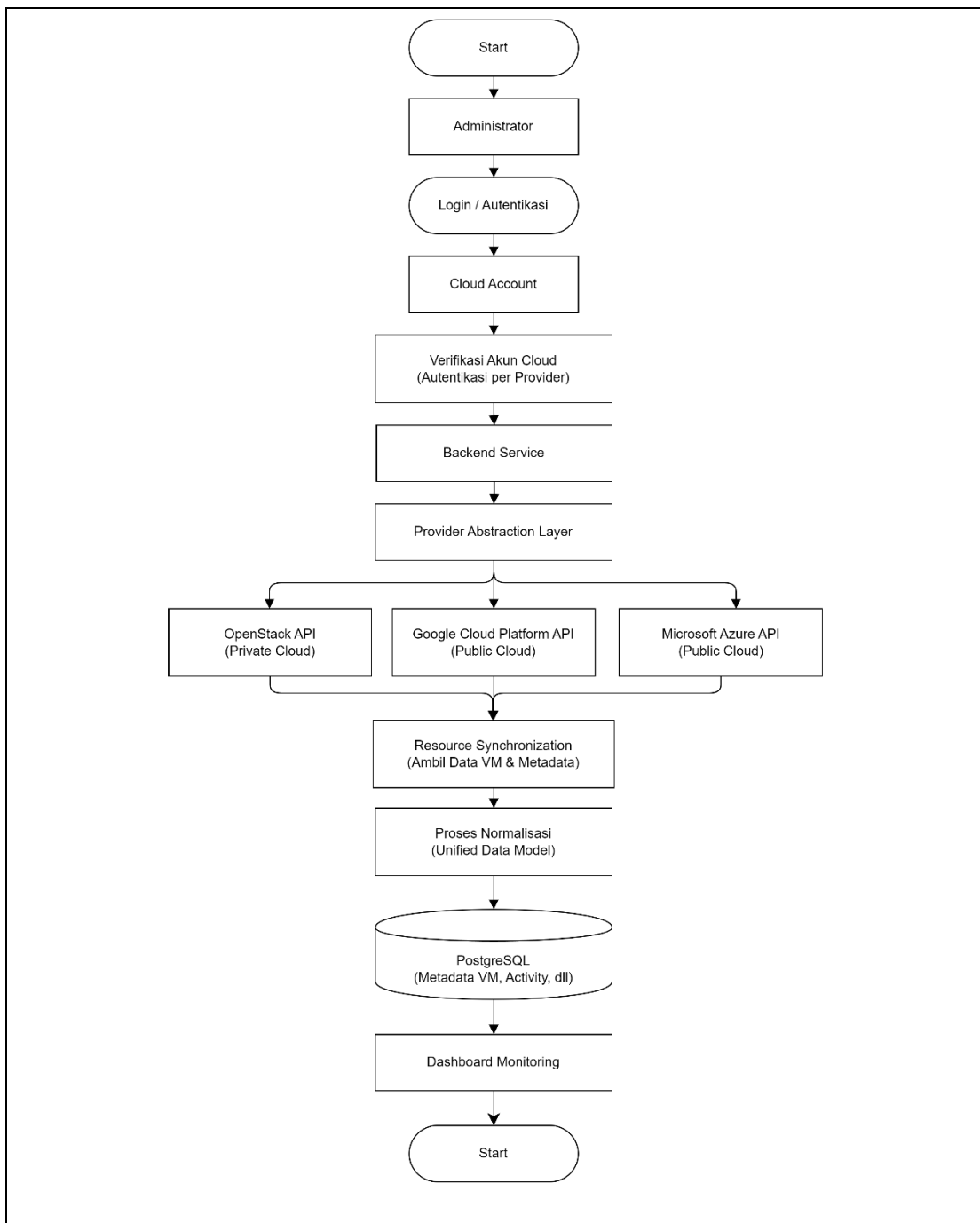


Gambar 2 Use Case Diagram Hybrid Cloud Monitoring dan Management

Diagram use case menunjukkan bahwa administrator merupakan aktor utama yang berinteraksi dengan sistem. Administrator dapat melakukan autentikasi, mengelola akun cloud, melakukan sinkronisasi sumber daya, memantau kondisi hybrid cloud melalui dashboard, melihat aktivitas sistem, serta mengelola siklus hidup virtual machine pada setiap cloud provider. Seluruh aktivitas tersebut diproses melalui backend sehingga administrator tidak perlu berinteraksi langsung dengan dashboard milik masing-masing cloud provider.

Sebelum akun cloud digunakan untuk proses monitoring maupun pengelolaan sumber daya, sistem melakukan proses autentikasi dan verifikasi kredensial sesuai mekanisme yang disediakan oleh masing-masing cloud provider. Tahapan ini bertujuan memastikan bahwa hanya akun yang valid dan memiliki hak akses yang sesuai yang dapat diintegrasikan ke dalam sistem. Penerapan mekanisme autentikasi yang aman merupakan salah satu aspek penting dalam pembangunan infrastruktur cloud karena berperan dalam menjaga kerahasiaan kredensial, mencegah akses tidak sah, serta meningkatkan keamanan layanan cloud (Alghofaili, Albattah, Alrajeh, Rassam, & Al-rimy, 2021)

Alur kerja sistem dimulai ketika administrator melakukan autentikasi pada aplikasi web. Setelah proses autentikasi berhasil, administrator dapat menambahkan akun cloud yang akan diverifikasi menggunakan mekanisme autentikasi masing-masing cloud provider. Backend kemudian melakukan proses Resource Synchronization untuk memperoleh informasi virtual machine beserta metadata dari setiap cloud provider. Informasi tersebut selanjutnya dinormalisasi ke dalam Unified Data Model sehingga data dari berbagai cloud provider dapat disimpan dalam struktur yang konsisten dan ditampilkan secara terpusat pada dashboard monitoring (Nzanzu, et al., 2022)



Gambar 3 Diagram Alur Monitoring dan Resource Synchronization

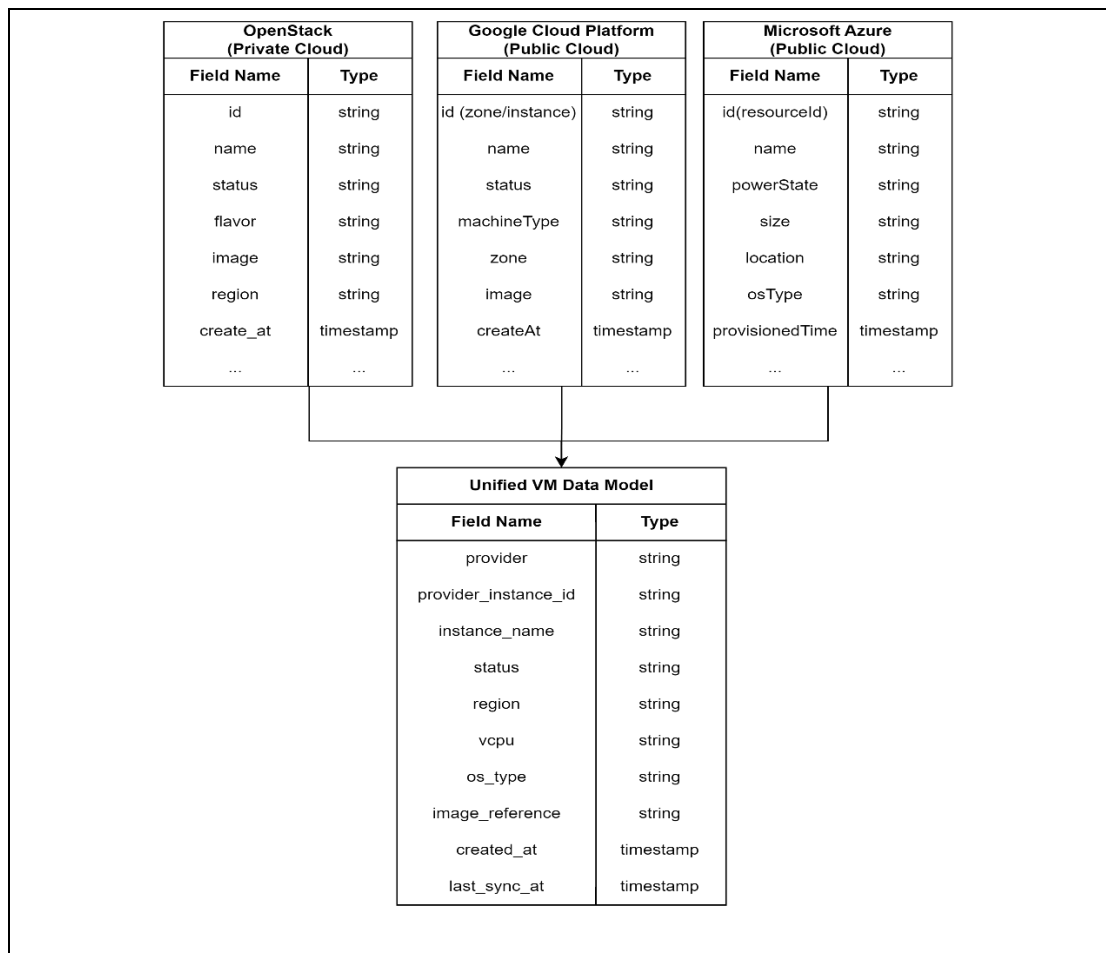
Proses monitoring dimulai ketika administrator melakukan autentikasi ke dalam aplikasi menggunakan akun yang telah terdaftar. Setelah proses autentikasi berhasil, administrator dapat menambahkan akun cloud yang akan dihubungkan dengan sistem. Setiap akun cloud diverifikasi menggunakan mekanisme autentikasi

yang disediakan oleh masing-masing cloud provider sebelum dapat digunakan untuk proses monitoring maupun manajemen sumber daya.

Selanjutnya, backend menjalankan proses Resource Synchronization dengan mengirimkan permintaan (*request*) ke setiap cloud provider melalui Provider Abstraction Layer. Lapisan ini menerjemahkan setiap permintaan menjadi format API yang sesuai dengan provider tujuan, kemudian mengambil informasi virtual machine beserta metadata yang diperlukan, seperti identitas instance, status, spesifikasi komputasi, lokasi (*region*), serta informasi pendukung lainnya. Pendekatan ini memungkinkan proses pengambilan data dilakukan secara seragam meskipun masing-masing cloud provider memiliki struktur API yang berbeda. (Andreazi, et al., 2021)

Informasi yang diperoleh dari setiap cloud provider selanjutnya diproses menggunakan mekanisme Unified Data Model. Pada tahap ini, metadata dari OpenStack, Google Cloud Platform, dan Microsoft Azure dinormalisasi ke dalam satu struktur data yang konsisten sehingga setiap virtual machine dapat direpresentasikan menggunakan atribut yang sama. Proses normalisasi tersebut mengurangi kompleksitas pengelolaan data serta meningkatkan interoperabilitas antar cloud provider. (Nzanzu, et al., 2022)

Setelah proses normalisasi selesai, data disimpan ke dalam PostgreSQL sebagai basis data utama. Penyimpanan metadata secara terpusat memungkinkan sistem menyajikan informasi monitoring tanpa harus selalu melakukan permintaan langsung ke setiap cloud provider, sehingga proses visualisasi data menjadi lebih cepat dan efisien (Kanthed, 2024).

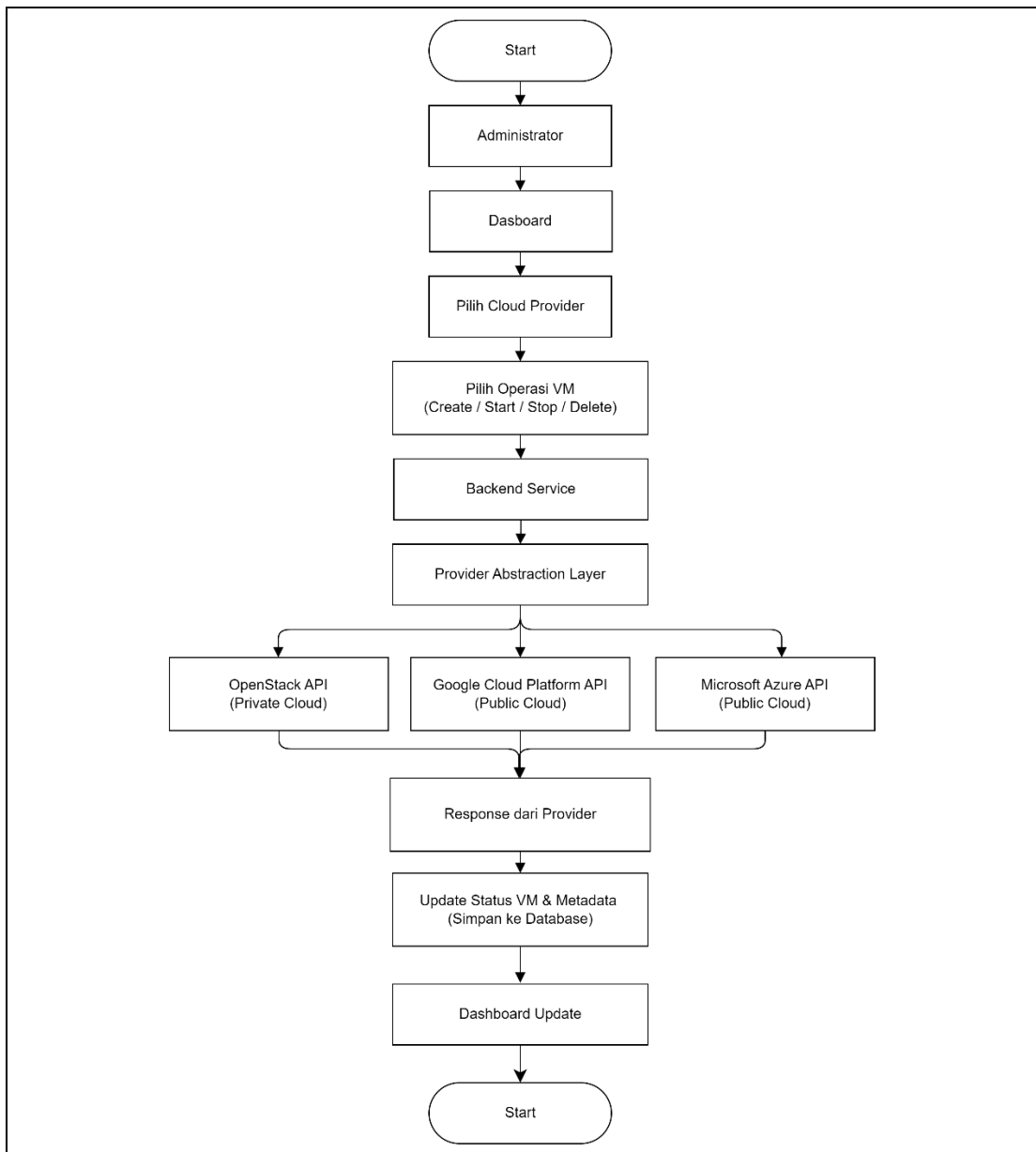


Gambar 4 Unified Data Model

Salah satu komponen utama yang diusulkan dalam penelitian ini adalah Unified Data Model, yaitu model data yang digunakan untuk menyamakan representasi sumber daya dari berbagai cloud provider. Meskipun OpenStack, Google Cloud Platform, dan Microsoft Azure memiliki struktur metadata yang berbeda, seluruh informasi tersebut dipetakan ke dalam atribut yang seragam sehingga dapat diproses oleh sistem tanpa bergantung pada implementasi masing-masing provider (Andreazi, et al., 2021).

Sebagai contoh, atribut seperti Instance ID, Instance Name, Status, Cloud Provider, Region, vCPU, Memory, Operating System, dan Creation Time direpresentasikan dalam satu struktur data yang sama. Dengan pendekatan ini, proses monitoring, pencarian data, penyaringan (*filtering*), maupun pengelolaan virtual machine dapat dilakukan secara konsisten tanpa memerlukan perlakuan khusus untuk

setiap cloud provider. Pendekatan normalisasi metadata seperti ini merupakan salah satu strategi yang direkomendasikan dalam pengelolaan lingkungan interconnected cloud infrastructures untuk meningkatkan interoperabilitas dan efisiensi sistem monitoring (Nzanzu, et al., 2022).

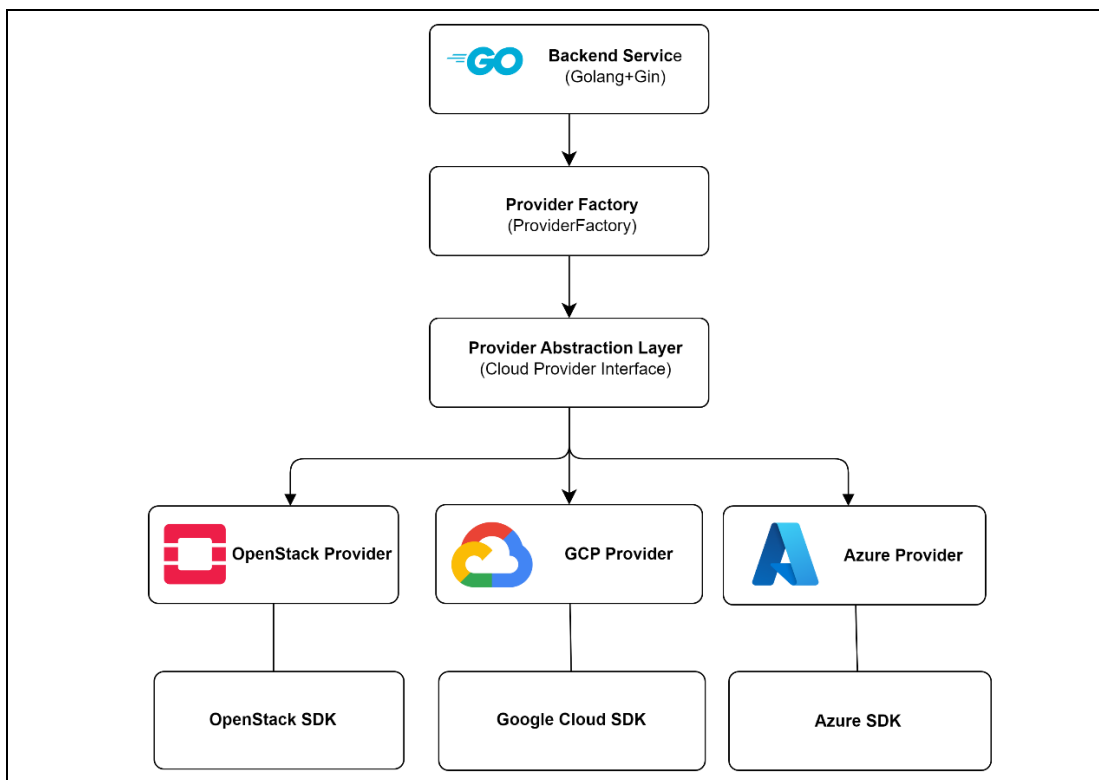


Gambar 5 Diagram Alur Virtual Machine Lifecycle Management

Alur pengelolaan Virtual Machine Lifecycle pada sistem yang diusulkan Administrator dapat melakukan operasi create, start, stop, maupun delete virtual

machine melalui dashboard tanpa perlu mengakses platform milik masing-masing cloud provider. Setiap permintaan diterima oleh Backend Service, kemudian diteruskan menuju Provider Abstraction Layer yang akan menerjemahkan permintaan tersebut menjadi format API yang sesuai dengan cloud provider tujuan. Pendekatan ini memungkinkan seluruh operasi manajemen virtual machine dilakukan melalui antarmuka yang seragam meskipun setiap cloud provider memiliki mekanisme API yang berbeda (Andreazi, et al., 2021).

Setelah operasi berhasil dijalankan, backend memperbarui metadata virtual machine pada basis data sehingga perubahan status instance dapat langsung ditampilkan pada dashboard monitoring. Dengan demikian, informasi yang disajikan kepada administrator selalu mencerminkan kondisi sumber daya terkini (Andreazi, et al., 2021).



Gambar 6 Struktur Provider Abstraction Layer

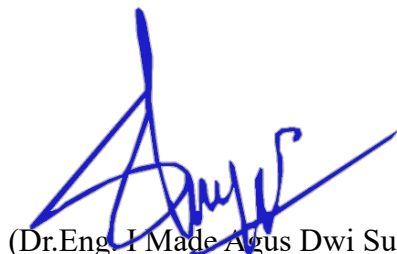
Provider Abstraction Layer yang digunakan sebagai lapisan integrasi antara backend dengan berbagai cloud provider. Komponen ini berfungsi menyembunyikan

perbedaan implementasi API pada OpenStack, Google Cloud Platform, dan Microsoft Azure sehingga backend tidak perlu berkomunikasi secara langsung dengan masing-masing provider (Andreazi, et al., 2021). Seluruh komunikasi dilakukan melalui Cloud Provider Interface yang menyediakan sekumpulan operasi umum, seperti Authenticate(), ListInstances(), CreateInstance(), StartInstance(), StopInstance(), dan DeleteInstance().

Setiap cloud provider memiliki implementasi yang berbeda terhadap operasi-operasi tersebut. Sebagai contoh, proses autentikasi pada OpenStack menggunakan Keystone, Google Cloud Platform menggunakan Service Account, sedangkan Microsoft Azure menggunakan Service Principal. Meskipun mekanisme autentikasi dan struktur API berbeda, seluruh implementasi mengikuti antarmuka yang sama sehingga backend dapat memanggil fungsi yang identik tanpa bergantung pada provider tertentu (Andreazi, et al., 2021). Pendekatan ini sejalan dengan konsep provider abstraction yang bertujuan meningkatkan interoperabilitas, modularitas, dan kemudahan pemeliharaan aplikasi multi-cloud

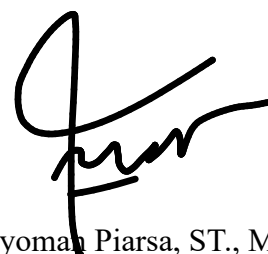
Dengan adanya Provider Abstraction Layer, penambahan cloud provider baru pada masa mendatang dapat dilakukan dengan mengembangkan implementasi provider yang sesuai tanpa memerlukan perubahan pada logika bisnis backend maupun antarmuka pengguna. Pendekatan ini mendukung pengembangan sistem yang lebih mudah diperluas (*extensible*) dan mengurangi ketergantungan terhadap implementasi API dari penyedia layanan cloud tertentu. (Li, Ma, Fang, He, & Li, 2025)

Calon Pembimbing 1,



(Dr.Eng. I Made Agus Dwi Suarjaya, ST, M.Eng)

Calon Pembimbing 2,



(Ir. I Nyoman Piarsa, ST., MT., IPM.)

4. Daftar Pustaka

- Alghofaili, Y., Albattah, A., Alrajeh, N., Rassam, M. A., & Al-rimy, B. A. (2021). Secure cloud infrastructure: A survey on issues, current solutions, and open challenges. *Applied Sciences*, 11(19), 9005. doi:<https://doi.org/10.3390/app11199005>
- Andreazi, G., Estrella, J., Bruschi, S., Immich, R., Guidoni, D., Alves Pereira Júnior, L., & Meneguette, R. M. (2021). MoHRiPA—An architecture for hybrid resources management of private cloud environments. *Sensors*, 21(20), 6857. doi:<https://doi.org/10.3390/s21206857>
- Kanthed, S. (2024). Monitoring of Cloud Computing Environments: Concepts, Solutions, Trends, and Future Directions. *International Journal on Science and Technology (IJSAT)*, 15(1), 1-16. doi:10.71097/ijst.v15.i1.2836
- Li, W., Ma, G., Fang, W., He, X., & Li, J. (2025). Multi-Cloud Management Architecture Design and Disaster Recovery Strategy for High Availability. *Journal of Cyber Security and Mobility*, 14(5), 1173-1198. doi:10.13052/jcsm2245-1439.1456
- Mamidala, J. V., Attipalli, A., Enokkaren, S. J., Bitkuri, V., Kendyala, R., & Kurma, J. (2025). A Survey on Hybrid and Multi-Cloud Environments: Integration Strategies, Challenges, and Future Directions. *International Journal of Humanities and Information Technology*, 5(02), 53-65. doi:<https://doi.org/10.21590/ijhit.05.02.08>
- Mansouri, Y., Prokhorenko, V., & Babar, M. A. (2020). An automated implementation of hybrid cloud for performance evaluation of distributed databases. *Journal of Network and Computer Applications*, 167, 102740. doi:<https://doi.org/10.1016/j.jnca.2020.102740>
- Merlo, T. R., Fard, F., & Hawamdeh, S. (2025). Cloud Computing's Impact on the Digital Transformation of the Enterprise: A Mixed-Methods Approach. *Sustainability*, 17(13), 5755. doi:<https://doi.org/10.3390/su17135755>

- Nzanzu, V. P., Adetiba, E., Badejo, J. A., Molo, M. J., Takenga, C., Noma-Osaghae, E., . . . Suraju, S. (2022). Monitoring and resource management taxonomy in interconnected cloud infrastructures: a survey. *TELKOMNIKA Telecommunication Computing Electronics and Control*, 20(2), 279-295. doi:10.12928/TELKOMNIKA.v20i2.20503
- Saxena, D., Gupta, R., & Singh, A. K. (2021). A Survey and Comparative Study on Multi-Cloud Architectures: Emerging Issues And Challenges For Cloud Federation. *arXiv*, 1-14. doi:https://doi.org/10.48550/arXiv.2108.12831
- Waseem, M., Ahmad, A., Liang, P., Akbar, M. A., Khan, A. A., Ahmad, I., . . . Mikkonen, T. (2024). Containerization in Multi-Cloud Environment: Roles, Strategies, Challenges, and Solutions for Effective Implementation. *arXiv*, 1-52. doi:https://doi.org/10.48550/arXiv.2403.12980